

Lecture 4

NLP: Text Vectorization and Word Embeddings

Advanced Topics in Artificial Intelligence · UFABC

Adapted from MIT 15.773 (Farias, Ramakrishnan) — OCW

Part 1 — Why NLP?

Lecture roadmap

Part 1 — Why NLP? Motivation, applications and historical arc

Part 2 — Text Pre-processing Pipeline Standardize → Tokenize → Index → Encode

Part 3 — Bag of Words Count encoding, multi-hot, limitations

Part 4 — Word Embeddings One-hot problems, Word2Vec, GloVe

Part 5 — Embeddings in Code `nn.Embedding` (PyTorch), `Embedding` (Keras)

Part 6 — Application Text classification with embeddings

Why NLP?

Human knowledge lives in natural language

- The Internet is, for the most part, **text**
- Human communication and cultural production are **text**
- Corporate documents, contracts, medical records are **text**

Imagine a system that could automatically read and "understand" all of this — extracting patterns, classifying, answering questions, generating content.

Applications in production



Classification

Sentiment, intent, routing



Extraction

Financials, form data



Summarization

Abstracts, bullet points, titles



Generation

Emails, reports, code



Q&A / RAG

Chatbots, semantic search



Translation

Multilingual, code-switching

Historical arc of NLP



Rules

up to 1990

- Formal grammars
- Manual syntax parsers
- Performance limited by rule scale



Statistical / ML

1990 – 2013

- Bag of Words + SVM
- Hidden Markov Models
- Logistic regression over n-grams



Neural Networks

2014 – 2017

- RNNs and LSTMs
- Word2Vec, GloVe
- Seq2seq with attention



Transformers

2017 – today

- "Attention is All You Need"
- BERT, GPT, T5, LLaMA
- General-purpose LLMs

"Every time I fire a linguist, the performance of the speech recognizer goes up." — Frederick Jelinek, IBM

Big picture: NLP as regression

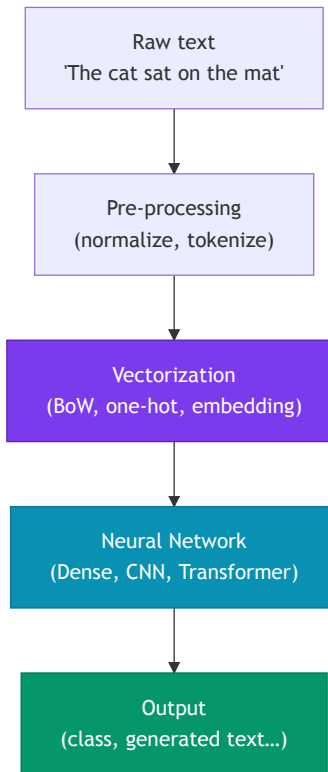
At its core, it's fancy regression

$$\hat{y} = f(\mathbf{x}, \theta)$$

- $\mathbf{x}$ = input text (token sequence)
- $\hat{y}$ = text, label, number, ...
- θ = neural network weights
- f = deep architecture (CNN, RNN, Transformer...)

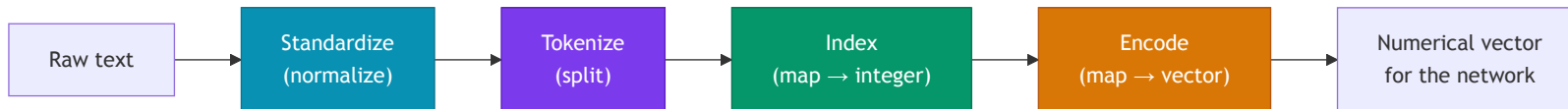
Central question of this lecture:

How do we represent $\mathbf{x}$ (text) as a numerical vector so the network can learn?



Part 2 — Text Pre-processing

Pre-processing pipeline



Standardize

- Lowercase
- Remove punctuation/accents
- *Stop words* (sometimes)
- Stemming/lemmatization (sometimes)

Tokenize

- Split into tokens
- Default: whitespace
- Alternative: subwords (BPE)
- N-grams (bigram, trigram...)

Index

- Assign unique integer to each token
- Builds the **vocabulary** $\mathcal{V}$
- Special token `<UNK>` for out-of-vocabulary words

Encode

- Map integer $\rightarrow$ vector
- Simplest: one-hot
- Better: word embedding

Pre-processing example

Original text:

```
"Hola! What do you picture when you think of traveling to Mexico? Sipping a real margarita while soaking up the sun on a laid-back beach in Puerto Vallarta?"
```

After standardize (lowercase, remove punctuation, stop words, stemming):

```
`hola picture think travel mexico sip real margarita soak sun  
laidback beach puerto vallarta`
```

After tokenize (by whitespace):

```
`["hola", "picture", "think", "travel", "mexico", "sip", "real",  
"margarita", "soak", "sun", "laidback", "beach", "puerto",  
"vallarta"]`
```

After index (vocabulary with 50 k tokens):

```
`[4231, 1807, 2943, 8821, 9012, 3344, 771, 18432, 6621, 488, 22301, 993, 19034, 19035]`
```

One-hot encoding

Idea: each vocabulary token gets a vector with 1 at its position and 0 everywhere else.

```
Vocabulary (|V| = 5):  cat, dog, fish, bird, mouse
```

```
cat   → [1, 0, 0, 0, 0]
```

```
dog   → [0, 1, 0, 0, 0]
```

```
fish  → [0, 0, 1, 0, 0]
```

```
bird  → [0, 0, 0, 1, 0]
```

```
mouse → [0, 0, 0, 0, 1]
```

- Vector dimension = $|\mathcal{V}|$ (vocabulary size)
- Special token `<UNK>` (index 0) for out-of-vocabulary words
- **Sparse** representation — only one non-zero element

Problem: for $|\mathcal{V}| = 50,000$, each token becomes a 50 k-dimensional vector.

✗ **Very high dimensionality** → many parameters, overfitting

✗ **No semantic similarity** — distance between any two one-hot vectors is always $\sqrt{2}$, regardless of meaning

```
from sklearn.preprocessing import LabelBinarizer
```

```
vocab = ["cat", "dog", "fish", "bird", "mouse"]
```

```
lb = LabelBinarizer()
```

```
lb.fit(vocab)
```

```
print(lb.transform(["cat", "fish"]))
```

```
# [[1 0 0 0 0]
```

```
#  [0 0 1 0 0]]
```

Part 3 — Bag of Words

From per-token to per-document vectors

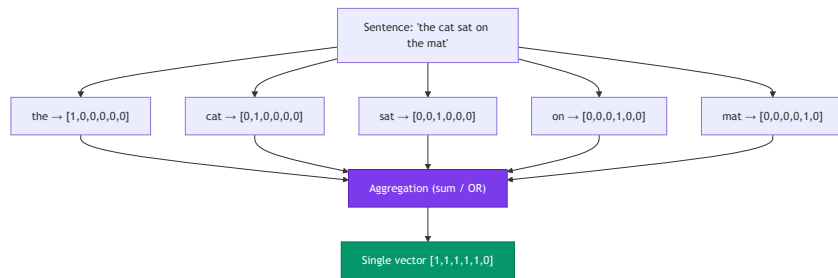
The problem: sentences have different lengths.

A 5-token sentence gives a $5 \times |\mathcal{V}|$ matrix.

A 20-token sentence gives $20 \times |\mathcal{V}|$.

Dense networks need **fixed-size input**.

Solution: aggregate (*pool*) all token one-hot vectors into a single vector of size $|\mathcal{V}|$.



Bag of Words — two variants

Count encoding (sum of one-hot vectors)

Counts how many times each token appears.

```
"the cat sat on the mat"  
"the mat belonged to the cat"  
  
vocab = [the, cat, sat, on, mat, belonged, to]  
  
Sentence 1: [2, 1, 1, 1, 1, 0, 0]  
Sentence 2: [2, 1, 0, 0, 1, 1, 1]
```

If "cat" appears 3×: that component = 3.

Multi-hot encoding (OR of one-hot vectors)

Indicates only whether the token is present (0 or 1).

```
"the cat cat cat sat"  
  
Count:      [1, 3, 1, 0, 0, 0, 0]  
Multi-hot: [1, 1, 1, 0, 0, 0, 0]
```

Multi-hot ignores frequency; useful when presence matters more than count.

Both are forms of **Bag of Words** — the name comes from treating text as a "bag" of words without order.

N-grams — capturing local context

Unigram: 1 token at a time (standard BoW)

Bigram: consecutive pairs of tokens

Trigram: consecutive triples of tokens

```
Sentence: "the cat sat on the mat"
```

```
Unigrams: ["the", "cat", "sat", "on", "the", "mat"]
```

```
Bigrams:  ["the cat", "cat sat",  
           "sat on", "on the", "the mat"]
```

```
Trigrams: ["the cat sat",  
           "cat sat on",  
           "sat on the",  
           "on the mat"]
```

Why use n-grams?

- ✓ Preserves some local context
- ✓ "not good" ≠ "good" (bigram captures negation)
- ✓ Simple, works well for text classification

- ✗ Vocabulary grows explosively: $|\mathcal{V}|^n$
- ✗ Still ignores long-range dependencies
- ✗ "good but not great" would need a 4-gram

```
from sklearn.feature_extraction.text import CountVectorizer  
cv = CountVectorizer(ngram_range=(1, 2))  
X = cv.fit_transform(["the cat sat on the mat"])
```

Bag of Words limitations

✗ Loses word order

"The dog bit the man"

and

"The man bit the dog"

have the **same** BoW vector.

✗ High dimensionality

$|\mathcal{V}|$ can be 50 k–500 k.

Every input has that dimension
regardless of sentence length.

Many parameters → overfitting,
slow.

✗ No semantics

"movie" and "film" are equidistant
from each other and from "banana".

BoW does not capture that they are
related words.

Summary: BoW is a useful, simple baseline, but for tasks requiring semantic understanding or long-range dependencies we need something better
— **Word Embeddings.**

Part 4 — Word Embeddings

The problem with one-hot representations

Problem 1: High dimensionality

Vocabulary of 50 k words → 50 k-dimensional vector.

Most elements are zero (**sparse**).

Problem 2: No semantic distance

```
"movie" → [1, 0, 0, 0, 0, ...]
"film" → [0, 1, 0, 0, 0, ...]
"banana" → [0, 0, 1, 0, 0, ...]

cosine("movie", "film") = 0
cosine("movie", "banana") = 0
# All words are equally "distant"!
```

The ideal solution

Dense and **compact** vectors (dimension 50–300) where **geometric distance** reflects **semantic distance**.

```
"movie" ≈ [0.2, 0.8, -0.3, ...] # compact
"film" ≈ [0.19, 0.79, -0.28, ...] # close to "movie"
"banana" ≈ [-0.6, 0.1, 0.9, ...] # far from both

cosine("movie", "film") ≈ 0.97 ✓
cosine("movie", "banana") ≈ 0.12 ✓
```

The intuition: "you shall know a word by the company it keeps"

"You shall know a word by the company it keeps."

— John Firth, linguist (1957)

Similar contexts → related words

"The acting in the _____ was superb."

- movie ✓
- film ✓
- musical ✓
- truck ✗
- banana ✗

Since "movie", "film" and "musical" appear in the **same contexts**, we infer they are semantically related.

Key insight

Count how often two tokens co-occur in the same contexts and learn vectors that **approximate** that co-occurrence matrix.

This is the foundation of two highly influential algorithms:

- **Word2Vec** (Mikolov et al., 2013) — local prediction
- **GloVe** (Pennington et al., 2014) — global co-occurrence

Word2Vec — two models

CBOW (*Continuous Bag of Words*)

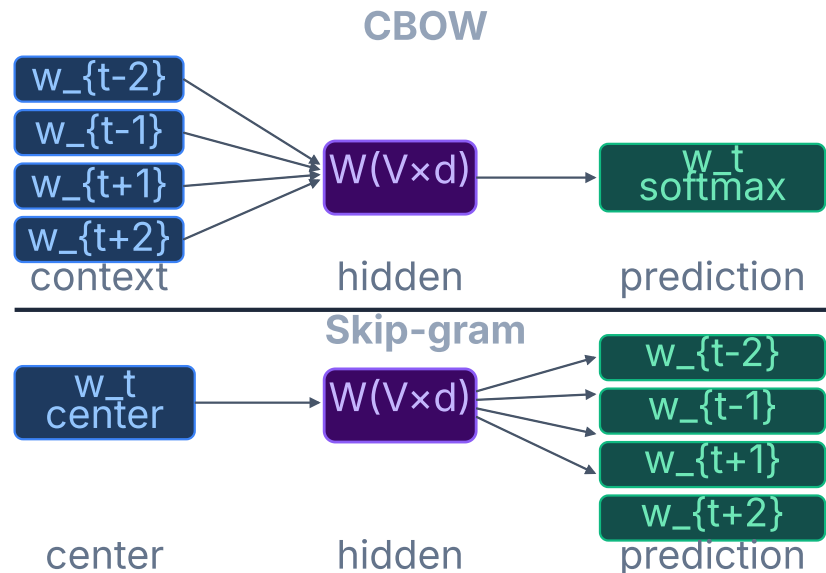
Given the context, predict the center word.

```
Context → Center word  
["The", "acting", "was", "superb"] → "in the ___"
```

Skip-gram

Given the center word, predict the context words.

```
Center word → Context  
"movie" → ["The", "acting", "was", "superb"]
```



💡 **Negative Sampling** — avoids softmax over $|V|$; classifies (w, ctx) as real or random using $k \approx 5-20$ negatives per step $\rightarrow O(k)$ instead of $O(|V|)$

Word2Vec — Negative Sampling

Problem: softmax over 50 k tokens at every step is costly — $O(|\mathcal{V}|)$.

Solution — Negative Sampling:

Instead of normalizing over the whole vocabulary, train a **binary classifier**:

$$P(D=1 \mid w, c) = \sigma(\mathbf{v}_c^\top \mathbf{v}_w)$$

"Did this (word, context) pair come from the real corpus?"

```
Positive pair: ("movie", "acting") → label 1
Negative pairs (k random samples):
("movie", "table") → label 0
("movie", "cloud") → label 0
...
```

Objective with negative sampling:

$$\mathcal{L} = \log \sigma(\mathbf{v}_c^\top \mathbf{v}_w) + \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n} [\log \sigma(-\mathbf{v}_{w_i}^\top \mathbf{v}_w)]$$

- ✓ Per-step complexity: $O(k)$ with $k \ll |\mathcal{V}|$ (typically $k = 5-20$)
- ✓ Empirically produces embeddings of similar quality to full softmax

More frequent words have a higher probability of being sampled as negatives (distribution $P_n(w) \propto f(w)^{3/4}$).

GloVe — Global Vectors

Intuition: Word2Vec uses local windows. GloVe uses **global co-occurrences** of the corpus.

Step 1: Count X_{ij} = how many times word j appears in the context of word i across the entire corpus.

	movie	film	banana	...
movie	[-, 8432, 12, ...]			
film	[8432, -, 9, ...]			
banana	[12, 9, -, ...]			

"movie" and "film" co-occur a lot; both rarely co-occur with "banana".

Step 2: Learn vectors that approximate the co-occurrence matrix.

Log-bilinear model:

$$\log X_{ij} = \mathbf{w}_i^\top \mathbf{w}_j + b_i + b_j$$

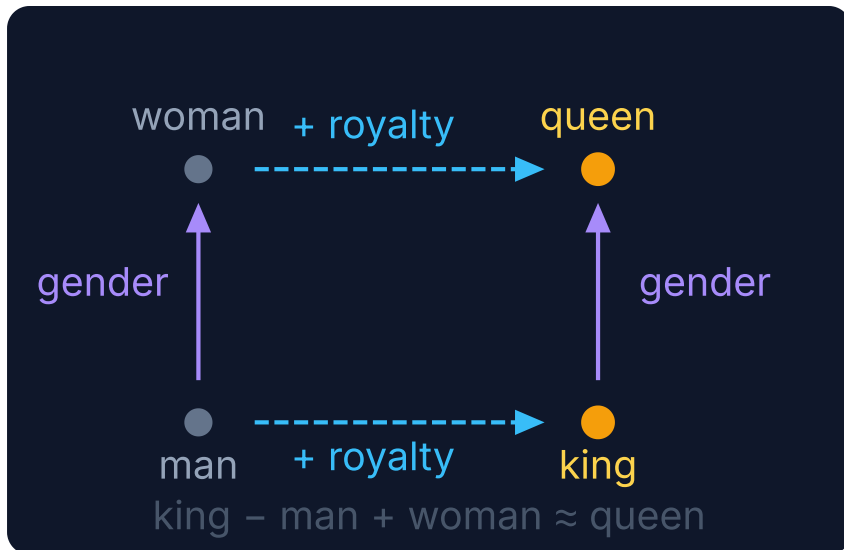
Objective (weighted least squares):

$$\mathcal{L} = \sum_{i,j} f(X_{ij}) (\mathbf{w}_i^\top \mathbf{w}_j + b_i + b_j - \log X_{ij})^2$$

where $f(X_{ij})$ is a weighting function that prevents very frequent pairs from dominating training.

At the end, biases b are discarded and only $\mathbf{w}$ is used.

Geometric properties of embeddings



2D projection — each dimension captures a latent concept that emerges from training.

Types of relations that emerge

Gender

king → queen · actor → actress · man → woman

Country → Capital

France → Paris · Brazil → Brasília · Japan → Tokyo

Superlative / Degree

good → better → best · big → bigger → biggest

Verb tense

run → ran · sing → sang · go → went

Antonyms

hot ↔ cold · fast ↔ slow · big ↔ small

⚠ These patterns emerge from the corpus — and also reflect its **biases** (Wikipedia, Common Crawl...).

FastText — Subword Embeddings

Problem with Word2Vec/GloVe: each word has a single vector
— out-of-vocabulary (OOV) words get no representation.

FastText idea (Bojanowski et al., 2017):

instead of one vector per word, learn vectors for **character n-grams**.

```
"where" → n-grams (n=3):
```

```
<wh, whe, her, ere, re>
```

```
+ the whole token <where>
```

```
vector("where") =  $\sum$  n-gram vectors
```

- ✅ **OOV:** "whereabouts" unseen → compose from known n-grams
- ✅ **Morphology:** "run", "running", "runner" share n-grams
- ✅ **Noise/typos:** "teh" ≈ "the" via overlapping n-grams

Comparison

	Word2Vec / GloVe	FastText
Unit	word	char n-gram
OOV	✗ no vector	✅ compose
Morphology	✗ ignored	✅ captured
Speed	✅ fast	⚠ slower
Size	✅ smaller	⚠ larger

In practice: FastText excels in **morphologically rich languages**

(Portuguese, German, Finnish) and noisy text (social media). Pre-trained vectors available for 157 languages at fasttext.cc.

Static vs contextual embeddings

Static embeddings

(Word2Vec, GloVe, FastText)

- One fixed vector per word, independent of context
- Fast to compute, low memory footprint
- Good for simple classification tasks

✗ "bank" (financial) has the same vector as "bank" (riverbank)

Contextual embeddings

(ELMo, BERT, GPT)

- Vector depends on all other tokens in the sentence
- Captures polysemy and long-range dependencies
- Foundation of modern LLMs

✓ "bank" generates different vectors in each context —
computed by a **Transformer** (Lecture 6!)

This lecture covers static embeddings. **Lecture 5** covers RNNs/LSTMs and **Lecture 6** covers Transformers and contextual embeddings.

Part 5 — Embeddings in Code

nn.Embedding — PyTorch

What it is: a lookup table — maps integer indices to dense vectors.

```
import torch
import torch.nn as nn

# vocab_size = vocabulary size
# embed_dim = embedding dimension
embed = nn.Embedding(
    num_embeddings=10_000, # |V|
    embedding_dim=128      # d
)

# Token indices for a batch of 2 sentences
# (zero-padded to max_len=5)
tokens = torch.tensor([
    [42, 871, 3, 0, 0],
    [7, 1024, 55, 810, 2]
]) # (batch=2, seq_len=5)

out = embed(tokens) # (2, 5, 128)
print(out.shape)    # torch.Size([2, 5, 128])
```

Loading pre-trained weights (GloVe)

```
import numpy as np

# Assumes glove_matrix: (|V|, d) pre-loaded
embed = nn.Embedding(
    num_embeddings=vocab_size,
    embedding_dim=100
)
embed.weight = nn.Parameter(
    torch.tensor(glove_matrix, dtype=torch.float32)
)

# Optional: freeze weights during fine-tuning
embed.weight.requires_grad = False
```

`nn.Embedding` is equivalent to an `nn.Linear` layer without bias, but with $O(1)$ lookup — it does not perform a full matrix multiplication over all sentences.

Embedding layer — Keras

Full pipeline with Keras

```
import tensorflow as tf
from tensorflow import keras

# 1. Vectorization (STI)
vectorizer = keras.layers.TextVectorization(
    max_tokens=10_000, # |V|
    output_mode='int', # returns indices
    output_sequence_length=64 # max_len
)
vectorizer.adapt(train_texts) # learns vocabulary

# 2. Embedding
embed = keras.layers.Embedding(
    input_dim=10_000, # |V|
    output_dim=128, # d
    mask_zero=True # ignore padding
)
```

Full model

```
inp = keras.Input(shape=(1,), dtype='string')
x = vectorizer(inp) # (batch, 64)
x = embed(x) # (batch, 64, 128)
x = keras.layers.GlobalAveragePooling1D()(x)
# mean over seq_len → (batch, 128)
x = keras.layers.Dense(64, activation='relu')(x)
out = keras.layers.Dense(num_classes,
    activation='softmax')(x)

model = keras.Model(inp, out)
model.compile(
    loss='sparse_categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
```

Global Average Pooling — why it works

After `Embedding`, each sentence is a **matrix ($T \times d$)**. We need a **fixed vector** for the dense layer.

```
"the cat sat" → Embedding
[[0.2, -0.1, 0.8, ...], ← "the"
 [0.7,  0.3, 0.1, ...], ← "cat"
 [0.4, -0.2, 0.6, ...]] ← "sat"   shape: (T, d)
```

```
GAP → mean over T (dim=1)
→ [0.43, 0.0, 0.5, ...]          shape: (d,)
```

Why does it work? GAP is equivalent to a **semantic BoW**: each word contributes its meaning vector, and the average captures the "mean semantic content" of the sentence. Unlike classic BoW (where "movie" and "cinema" are orthogonal vectors), here semantically similar words contribute similar vectors — the result aggregates semantics, not just counts.

Alternatives to GlobalAveragePooling

Flatten — concatenates all vectors into $(T \times d)$, requires fixed length.

GlobalMaxPooling1D — maximum per dimension, preserves strongest activations.

[CLS] token / attention — Transformer strategies (Lecture 6). BERT uses `[CLS]`.

Shared limitation: none of these strategies preserve token **order**. For that, we need RNNs or Transformers.

Part 6 — Application: Text Classification

Musical genre classification

BoW + Embedding architecture

Task: given a song lyric excerpt, classify it as hip-hop, pop or rock.

*"I grew up on the crime side, the New York Times
side... Had secondhands, Mom's bounced on old man..."* →
hip-hop 🎤

*"I walked through the door with you But something
about it felt like home somehow..."* → **pop** 🎵

Full code — PyTorch

```
import torch, torch.nn as nn, torch.optim as optim

class TextClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_classes):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.dropout = nn.Dropout(0.3)
        self.fc1 = nn.Linear(embed_dim, 64)
        self.fc2 = nn.Linear(64, num_classes)

    def forward(self, x):          # x: (batch, seq_len)
        x = self.embed(x)          # (batch, seq_len, embed_dim)
        x = x.mean(dim=1)          # GAP: (batch, embed_dim)
        x = self.dropout(x)
        x = torch.relu(self.fc1(x))
        return self.fc2(x)         # (batch, num_classes)

model = TextClassifier(vocab_size=10_000, embed_dim=64, num_classes=3)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(10):
    for tokens, labels in train_loader:
        logits = model(tokens)
```

Comparison: BoW vs Embeddings

	BoW (count/multi-hot)	Embedding + GAP
Representation	Sparse ($ V $ -dimensional)	Dense (d -dim, $d \ll V $)
Semantics	✗ No semantic distance	✓ Similar words are close
Word order	✗ Ignored	✗ Ignored (with GAP)
Parameters	None (engineered)	$ V \times d$ (learned)
Transfer	Only with external embeddings	✓ Pre-train on large corpus
Cost	Low	Moderate

To capture word order and long-range dependencies → sequential architectures: RNNs and LSTMs (**Lecture 5**), and ultimately Transformers (**Lecture 6**).

Summary

Pre-processing

- S→T→I→E pipeline
- Standardization, tokenization, vocabulary
- One-hot encoding: sparse, no semantics

Bag of Words

- Count and multi-hot encoding
- N-grams for local context
- Limitations: order lost, high dimensionality

Word Embeddings

- Word2Vec (skip-gram, negative sampling)
- GloVe (global co-occurrence, least squares)
- Dense vectors with semantic distance

In code

- `nn.Embedding` (PyTorch)
- `Embedding` + `TextVectorization` (Keras)
- `GlobalAveragePooling1D` as weighted BoW

Remaining limitations

- No order with GAP
- Static embeddings: polysemy
- → We need Transformers!

Lecture 5 — RNNs and LSTMs

- Hidden state and sequential processing
- BPTT and the vanishing gradient problem
- LSTMs and GRUs as solutions

Next lecture

Lecture 5 — RNNs and LSTMs

- Hidden state and sequential processing
- BPTT and the vanishing gradient problem
- LSTM: memory cells and gates
- GRU: more efficient variant
- Applications: classification and seq2seq

For this week

- Notebook `lec04-embeddings.ipynb` :
 - Pre-processing pipeline with Keras/PyTorch
 - Train embeddings from scratch vs pre-trained GloVe
 - Musical genre classification
 - Embedding visualization with t-SNE

Thank you! Questions?

CCM-109 · Deep Learning · UFABC

Adapted from MIT 15.773 Hands-on Deep Learning (Farias & Ramakrishnan, 2024)